

Client and Server

Memory Caching Options

Architecture Component #7

Server Side Caching: NodeJS & DotNet have direct access to Server Memory.

AI Mode All Videos Shopping Images More ▾

```
const ramDiskPath = fs.mkdtempSync(path.join(tmpDir, 'js-ramdisk-'));

fs.writeFileSync(path.join(ramDiskPath, 'data.json'), JSON.stringify({ speed: "f.
```

Use code with caution.



💡 Comparison: Approaches to JavaScript RAM Disks

Approach	Where It Lives	Primary Use Case	Performance Benefit
System tmpfs Mount	OS-Level RAM Kernel	Speeding up <code>npm install</code> , Webpack/Vite bundlers, and heavy CI/CD pipelines.	Highest for existing CLI tools; eliminates actual disk writes completely.
memfs Library	Node.js Process Heap Memory	Unit testing, custom caching layers, and running bundlers entirely inside a script.	Fastest execution, but limited strictly to that individual Node program.
OS tmpdir()	System Managed /tmp Storage	Safely writing temporary worker files or logs without cluttering the persistent drive.	High , depending on whether the host OS mounts <code>/tmp</code> as a RAM partition.

2. Programmatic Virtual Memory Disks in Node.js

If you are writing a JavaScript script and want a purely programmatic data space that lives in memory (and mimics a real filesystem), you should use an **in-memory virtual filesystem**. Node.js does not have a native API to format real hardware RAM disks, but it does allow you to replace or mock the file system (`fs`) module. [🔗](#)

Option A: Using `memfs` (Recommended)

The [memfs GitHub Repository](#) provides a complete in-memory filesystem implementation that mirrors Node's native `fs` API exactly.

javascript

```
import { volume } from 'memfs';

// Setup file structure strictly in system memory
volume.fromJSON({
  '/dir/file.txt': 'Hello from your JavaScript RAM disk!',
});

// Read directly from the in-memory store
const data = volume.readFileSync('/dir/file.txt', 'utf8');
console.log(data); // Output: Hello from your Javascript RAM disk!
```

Use code with caution.



Option B: Native Node.js Virtual Temp Dirs

If you just need a ultra-fast temporary directory that utilizes the operating system's native `/tmp` or `/dev/shm` RAM storage structures, you can use the built-in `os` module: [🔗](#) [GeeksforGeeks](#)

javascript

```
import os from 'os';
import fs from 'fs';
import path from 'path';

// Resolves to the OS temp directory (often a tmpfs RAM disk on modern Linux dis
const tmpDir = os.tmpdir();
```



Ask anything



Server Side Any Data Structure is Possible in Practice to Memory or Temp Disks.

Client Side A Few Options

Browsers Do Not Have Access to Disk...

- 1) Except For Browser Cookies
- 2) Session Storage
- 3) Local Storage.

These all create temporary files in the Browsers local directory on the hard drive.

#4) ALTERNATIVELY... JAVASCRIPT CAN MAKE A CALL TO BROWSER MEMORY, AND CREATE A RAM DISK FOR TEMPORARY SESSION ACCESS..... (DISKLESS WORKSTATION MODEL).....

1. The Pure Memory Approach: JavaScript State & Blobs

For simple scenarios where you need to generate, modify, and access file-like data ultra-fast without hitting physical storage, you can use native **Blobs** and **Object URLs**.

```
javascript
// 1. Create data and store it strictly in browser memory
const fileContent = new Blob(["Hello from browser RAM disk!"], { type: "text/plain" });

// 2. Generate an in-memory virtual URL (pointer to RAM)
const ramDiskUrl = URL.createObjectURL(fileContent);

console.log(ramDiskUrl); // Outputs: blob:https://yourwebsite.com...

// 3. You can now pass this URL to <img>, <video>, <a> tags, or fetch it
fetch(ramDiskUrl)
  .then(res => res.text())
  .then(text => console.log(text)); // "Hello from browser RAM disk!"

// 4. Clean up memory when done
URL.revokeObjectURL(ramDiskUrl);
```

Use code with caution.

DIM A BLOB

2. The Professional Approach: WebAssembly & Lightning-FS

If you are porting a complex app to the browser (like an IDE, a video editor, or a Git client) and you need a virtual, in-memory filesystem that behaves exactly like a real OS drive, you should use **@isomorphic-git/lightning-fs** or **memfs** bundled for the browser.

Lightning-FS allows you to create a completely isolated file system that lives purely in memory:

```
javascript
import FS from '@isomorphic-git/lightning-fs';

// Initialize a pure, non-persistent in-memory filesystem
const fs = new FS('my-browser-ramdisk', { wipe: true });

// Write files at near-instant memory speeds
await fs.promises.writeFile('/test.txt', 'Virtual drive data');

// Read files back
const data = await fs.promises.readFile('/test.txt', 'utf8');
console.log(data); // "Virtual drive data"
```

Use code with caution.

Web
Assembly

3. The Browser Native Alternative: Origin Private File System (OPFS)

If your goal is **maximum speed** but you don't necessarily want the data to vanish on a page refresh, you should use the **Origin Private File System (OPFS)**.

While OPFS is backed by actual physical storage rather than pure RAM, browsers optimize it aggressively. When accessed inside a **Web Worker**, it provides access to an exclusive "Access Handle" that allows raw, binary, synchronous, and incredibly fast read/write operations that rival memory speeds.

```
javascript
// Inside a Web Worker for maximum performance:
const root = await navigator.storage.getDirectory();
const fileHandle = await root.getFileHandle("speed-test.bin", { create: true });

// Get the high-performance synchronous access handle
const accessHandle = await fileHandle.createWritable({ keepExistingData: false });

const encoder = new TextEncoder();
const buffer = encoder.encode("high-speed browser i/o");

// Write directly into the browser's optimized storage sandbox
await accessHandle.write(buffer);
await accessHandle.close();
```

Use code with caution.

OPFS

Chrome/Mozilla Implementation of Cookies To SQLite

AI MODE All Videos Shopping Images More ▾

where are cookies stored in chrome?

Google Chrome stores cookies **inside an SQLite database file simply named cookies**. In modern versions of Chrome, this file is located inside a **Network** subfolder of your local user profile directory. [Stack Overflow +1](#)

Because Chrome **encrypts cookie values** using system-level cryptographic keys (like DPAPI on Windows or Keychain on macOS), you cannot read the data just by opening the file in a text editor. [Stack Overflow +1](#)

Physical File Paths by Operating System

To find the file on your machine, close Chrome completely and navigate to the following directory depending on your OS (make sure to enable "Show Hidden Files" in your OS settings): [🔗](#)

Windows:

```
%LOCALAPPDATA%\Google\Chrome\User Data\Default\Network\
(Or C:\Users\<Your_Username>\AppData\Local\Google\Chrome\User
Data\Default\Network\ )
```

macOS:

```
~/Library/Application Support/Google/Chrome/Default/Cookies
```

Linux:

```
~/.config/google-chrome/Default/Network/Cookies (or ~/.config/chromium/... if
using open-source Chromium) Stack Overflow +2
```

*Note: If you use Chrome profiles, replace **default** in the path with your specific profile folder name (e.g., **Profile 1**).*

AI Mode All Videos Shopping Images More ▾

where does mozilla store cookies

Mozilla Firefox stores all cookies **inside a single SQLite database file named cookies.sqlite**. This file is located inside your unique, randomized Firefox user profile folder. [Stack Overflow +1](#)

Unlike Google Chrome (which forcibly encrypts cookie values natively), Firefox stores cookies in **plain text** within this SQLite database. Anyone with access to the file can extract your session tokens unless you have a Master Password / Primary Password enabled to protect the profile. [Unix & Linux Stack Exchange +1](#)

Physical File Paths by Operating System

To find the file, close Firefox completely and navigate to your specific profile directory. Replace **<profile_folder>** with your actual profile name (which usually looks like a random string of characters ending in **.default** or **.default-release**). [Stack Overflow +1](#)

Windows:

```
%APPDATA%\Mozilla\Firefox\Profiles\<profile_folder>\cookies.sqlite
(Or C:\Users\<Your_Username>\AppData\Roaming\Mozilla\Firefox\Profiles\...)
```

macOS:

```
~/Library/Application Support/Firefox/Profiles/<profile_folder>/cookies.sqlite
```

Linux:

```
~/.mozilla/firefox/<profile_folder>/cookies.sqlite Foxton Forensics +3
```

Shortcut: If you are unsure what your profile folder name is, type **about:profiles** directly into the Firefox address bar. Look for the profile marked as **Profile: Default** and click the **Open Folder** button next to the "Root Directory" path. [Stack Overflow](#)

Inside the Database File

Because it is a standard SQLite database, you can open **cookies.sqlite** using any database viewer (like [DB Browser for SQLite](#)). [Stack Overflow](#)

All your active browser cookies live inside a table called **moz_cookies**. The schema of this

Super User

Where does Firefox keep cookies? - Super User
Feb 8, 2012 — all cookies are stored located in the Firefox profile folder. Under the Application Basics section, click...

Unix & Linux Stack Exchange

Where does Firefox store its cookies on Linux?
Firefox stores cookies in sqLite database ~/.mozilla/firefox/cookies. can do: cd...

Ask Ubuntu

Where do browsers save cookies and data?
You can use the Nautilus file manager to browse these directories. 1. Selecting...

Show all

OPERA AND EDGE USE CHROMES ENCRYPTED MODEL...

Cookies in Transit... Safari

does safari encrypt cookies?

Short answer: Safari does **not** encrypt cookies in transit by itself, but Safari cookie data stored on the device is protected by the operating system and browser storage mechanisms. Whether a cookie is encrypted while traveling across the network depends on whether the website uses HTTPS/TLS. apple

A few distinctions:

Cookies in transit

- If the website uses HTTPS, the cookie is transmitted inside the TLS-encrypted connection. Anyone sniffing the network cannot read the cookie contents. apple
- If the website uses HTTP, cookies are sent in clear text and can potentially be intercepted. (Modern sites should use HTTPS.)

Cookies stored on macOS/iOS

- Safari stores cookie and website data locally.
- Apple protects sensitive stored data using operating system security features, including encryption mechanisms used throughout the platform. Apple's keychain system encrypts stored secrets using AES-256-GCM and Secure Enclave-backed protection where applicable. apple

Important security note

A cookie's value itself is **usually not encrypted by the browser**. If you open Developer Tools on a website you're logged into, you'll typically see the cookie value in plain text. The browser stores and sends it as provided by the server.

Common patterns are:

```
1 SessionID=ABCD1234
```